

claude-windows / 2ba49afa-99f4-41f2-86a2-27a289d432f3

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\2ba49afa-99f4-41f2-86a2-27a289d432f3\subagents\workflows\wf_a48e6aa3-748\agent-a548326f7da4a476f.jsonl`  
- SHA-256: `b265d6d04620dc526388bed07e774445312c49d468311988edc26c37735b6ba2`  
- Source modified: `2026-06-21T16:35:51+00:00`  
- Imported at: `2026-07-05T16:48:28+00:00`  
- project: `wf_a48e6aa3-748`  
- session_id: `2ba49afa-99f4-41f2-86a2-27a289d432f3`
```

Transcript

1. user (2026-06-21T16:31:36.793Z)

Review T4 for ENGINE-CONTRACT FIDELITY. T4 (North Star keystone): in-UI local file UPLOAD wired to the engine's chunked-upload endpoints.

Files under C:/Users/avidu/Projects/khelsutra/web/src:

- api/client.ts: uploadInit/uploadPart(raw octet-stream PUT)/uploadComplete/uploadAbort + uploadFile(file,{cid,signal,onProgress}) orchestrator (init -> slice into max_part_mb parts -> sequential PUT 0,1,2 -> complete; best-effort DELETE on failure/abort).
- api/types.ts: UploadInit/Part/CompleteResponse.
- hooks/useUpload.ts: start(file) -> uploading{pct} -> onComplete(serverPath) | error(classifyError); rejects 0-byte; AbortController cancel/unmount.
- components/IngestPanel.tsx: file picker (primary) + the existing gs:// URI form (secondary); on upload complete -> useIngestJob.submit(path) (the SAME process->poll->onLoaded back-half); upload % progressbar; A8 chips reused.
- catalogs en/kn/hi: +ingest.chooseFile/uploadAria/fileHint/orLink/uploading.

Engine contract at C:/Users/avidu/Projects/badminton-highlight-indexer/backend/api/uploads.py. 150 web tests green; tsc + build clean.

READ the engine contract at C:/Users/avidu/Projects/badminton-highlight-indexer/backend/api/uploads.py carefully, then

C:/Users/avidu/Projects/khelsutra/web/src/api/client.ts. Verify the SPA matches EXACTLY:

1. init: POST /api/upload/init {filename, total_size_bytes} -> {upload_id, max_part_mb, next_part}. Does the SPA send the right body keys? Does it honor the RETURNED max_part_mb (not a hardcoded size)?
2. part: PUT /api/upload/{id}/part/{index} with RAW bytes (request.stream()), strictly sequential, per-part <= max_part_mb. Does the SPA's slice size ever EXCEED the engine's part_bytes limit (which 413s + aborts the whole upload)? (Engine part_bytes = max_part_mb*1024*1024; SPA partBytes = max_part_mb*1024*1024 ? equal; a full part == limit, is that > or <= ? confirm the engine uses > not >=.)
3. complete: POST /api/upload/{id}/complete {total_parts} -> {path, video_id, size_bytes}. Does the SPA send total_parts == the number of parts actually PUT? (Engine 409s if mismatch.)
4. The returned {path} is fed to POST /api/process {video_path: path}. Confirm /api/process accepts a host path in DEFAULT mode. NOTE any HOSTED-mode caveat (allowed_video_root must contain upload_dir) as a T2/deployment note, not a T4 blocker.
5. Size limits: init returns 413 if total_size_bytes > max_upload. Does the SPA surface that cleanly (friendly message)? Is there any client-side guard worth adding (e.g. reject 0-byte ? already done)?

Be precise (cite uploads.py line numbers). Empty findings if the contract matches.

2. assistant (2026-06-21T16:31:40.559Z)

I'll review T4 for engine-contract fidelity. Let me start by reading the engine contract and the SPA client carefully.

3. user (2026-06-21T16:31:43.987Z)

Your configuration specifies to merge with the ref 'refs/heads/serving/exposure-safety-preflight' from the remote, but no such ref was fetched.

4. user (2026-06-21T16:31:43.995Z)

```
1      """Chunked upload (B2, PR-2) ? extracted from backend.main into its own router.
2
3      Browser uploads arrive in sequential parts ? security.max_part_mb (free-tier edge
4      proxies cap request bodies at ~100 MB ? HOSTING_PLAN $2), are assembled server-side
5      under security.upload_dir, then submitted to /api/process by the returned path.
6      Upload state is in-process by design (this is the single-box alpha): a server restart
7      aborts partial uploads and the client re-inits. Per-user partitioning of upload_dir
8      arrives with PR-3 identity scoping.
9
10     The startup config lives on ``backend.main.global_config`` (set ONLY in ``main()``).
11     ``_upload_cfg`` resolves it through the ``backend.main`` module object so it always
12     reads the live value the server set at startup (and so tests that monkeypatch
13     ``backend.main.global_config`` flow through unchanged).
14     """
15     import os
16     import threading
17     import uuid
18     from typing import Any, Dict, Optional
19
20     from fastapi import APIRouter, HTTPException, Request
21     from pydantic import BaseModel
22
23     from backend.utils.hashing import compute_video_id # canonical file-identity hashing
24     from backend.utils.paths import safe_output_filename
25
26     router = APIRouter()
27
28     _uploads: Dict[str, Dict[str, Any]] = {}
29     _uploads_lock = threading.Lock()
30
31
32     class UploadInitRequest(BaseModel):
33         filename: str
34         total_size_bytes: Optional[int] = None
35
36
37     class UploadCompleteRequest(BaseModel):
38         total_parts: int
39
40
41     def _upload_cfg():
42         # Resolved lazily (import inside the function) to read the LIVE startup config that
43         # main() sets on the module global, and to avoid a circular import at module load
44         # (main.py imports this router; this module must not import main at import time).
45         import backend.main as _main
46         sec = getattr(_main.global_config, "security", None)
47         upload_dir = getattr(sec, "upload_dir", None) or "output/uploads"
48         max_bytes = int(getattr(sec, "max_upload_mb", 4096) or 4096) * 1024 * 1024
49         part_bytes = int(getattr(sec, "max_part_mb", 95) or 95) * 1024 * 1024
50         exts = [e.lower().lstrip(".") for e in (getattr(sec, "allowed_upload_extensions",
51         None) or ["mp4", "mov"])]
52         return upload_dir, max_bytes, part_bytes, exts
53
54     def _abort_upload(upload_id: str) -> None:
55         with _uploads_lock:
56             st = _uploads.pop(upload_id, None)
57             if st:
58                 try:
```

```

59         os.remove(st["tmp_path"])
60     except OSError:
61         pass
62
63
64 @router.post("/api/upload/init")
65 def upload_init(req: UploadInitRequest):
66     """Start a chunked upload. Returns upload_id; PUT sequential parts next."""
67     upload_dir, max_bytes, part_bytes, exts = _upload_cfg()
68     try:
69         name = safe_output_filename(req.filename)
70     except ValueError as e:
71         raise HTTPException(status_code=400, detail=str(e)) from e
72     ext = os.path.splitext(name)[1].lower().lstrip(".")
73     if ext not in exts:
74         raise HTTPException(status_code=400,
75                             detail=f"Extension '{ext}' not allowed. Allowed:
{sorted(exts)}")
76     if req.total_size_bytes is not None and req.total_size_bytes > max_bytes:
77         raise HTTPException(status_code=413,
78                             detail=f"File exceeds the {max_bytes // (1024 * 1024)} MB
upload limit.")
79     os.makedirs(upload_dir, exist_ok=True)
80     upload_id = uuid.uuid4().hex
81     tmp_path = os.path.join(upload_dir, f".partial-{upload_id}")
82     open(tmp_path, "wb").close()
83     with _uploads_lock:
84         _uploads[upload_id] = {"filename": name, "tmp_path": tmp_path, "parts": 0,
"bytes": 0}
85     return {"upload_id": upload_id, "max_part_mb": part_bytes // (1024 * 1024),
86           "next_part": f"/api/upload/{upload_id}/part/0"}
87
88
89 @router.put("/api/upload/{upload_id}/part/{index}")
90 async def upload_part(upload_id: str, index: int, request: Request):
91     """Append one part (raw bytes body). Parts are strictly sequential (0,1,2,...)."""
92     _, max_bytes, part_bytes, _ = _upload_cfg()
93     with _uploads_lock:
94         st = _uploads.get(upload_id)
95         if st is None:
96             raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
97         if index != st["parts"]:
98             raise HTTPException(status_code=409,
99                                 detail=f"Out-of-order part: expected {st['parts']}, got
{index} (parts are sequential)")
100
101         received = 0
102         overflow = None
103         with open(st["tmp_path"], "ab") as fh:
104             async for chunk in request.stream():
105                 received += len(chunk)
106                 if received > part_bytes:
107                     overflow = f"Part exceeds the {part_bytes // (1024 * 1024)} MB per-part
limit."
108                     break
109                 if st["bytes"] + received > max_bytes:
110                     overflow = f"Upload exceeds the {max_bytes // (1024 * 1024)} MB total
limit."
111                     break
112                 fh.write(chunk)
113         if overflow:
114             _abort_upload(upload_id)
115             raise HTTPException(status_code=413, detail=overflow + " Upload aborted ? re-
init to retry.")
116

```

```

117         with _uploads_lock:
118             st["parts"] += 1
119             st["bytes"] += received
120         return {"upload_id": upload_id, "received_parts": st["parts"], "received_bytes":
st["bytes"]}
121
122
123     @router.post("/api/upload/{upload_id}/complete")
124     def upload_complete(upload_id: str, req: UploadCompleteRequest):
125         """Finalize: verify part count, move into place, return {path, video_id} for
/api/process."""
126         upload_dir, _, _, _ = _upload_cfg()
127         with _uploads_lock:
128             st = _uploads.get(upload_id)
129         if st is None:
130             raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
131         if st["parts"] == 0 or st["parts"] != req.total_parts:
132             raise HTTPException(status_code=409,
133                                 detail=f"Expected {req.total_parts} part(s), received
{st['parts']}.")
134         final_path = os.path.join(upload_dir, st["filename"])
135         if os.path.exists(final_path): # never overwrite an earlier upload with the same
name
136             stem, ext = os.path.splitext(st["filename"])
137             final_path = os.path.join(upload_dir, f"{stem}-{upload_id[:8]}{ext}")
138         os.replace(st["tmp_path"], final_path)
139         with _uploads_lock:
140             _uploads.pop(upload_id, None)
141         video_id = compute_video_id(final_path)
142         return {"path": os.path.abspath(final_path), "video_id": video_id,
143               "size_bytes": os.path.getsize(final_path),
144               "next": "POST /api/process with this path"}
145
146
147     @router.delete("/api/upload/{upload_id}")
148     def upload_abort(upload_id: str):
149         """Abort an in-flight upload and delete its partial file."""
150         with _uploads_lock:
151             known = upload_id in _uploads
152         if not known:
153             raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
154         _abort_upload(upload_id)
155         return {"status": "aborted", "upload_id": upload_id}
156

```

5. user (2026-06-21T16:31:43.996Z)

```

1         // Typed client for the engine FastAPI. SAME-ORIGIN relative `/api/*` (the appliance
serves
2         // UI + engine behind one origin; in dev Vite proxies `/api` ? :8000). Every call
carries a
3         // correlation id (header + logs) so a journey is traceable end to end.
4         import { log, newCorrelationId } from "../lib/log";
5         import type {
6             RallySegment,
7             QueryResponse,
8             CompileResponse,
9             EngineConfig,
10            ProcessResponse,
11            JobStatus,
12            UploadInitResponse,
13            UploadPartResponse,
14            UploadCompleteResponse,
15        } from "./types";
16

```

```

17     const BASE = "/api";
18
19     export class ApiError extends Error {
20         readonly cid?: string;
21         readonly status?: number;
22         readonly body?: string;
23         constructor(message: string, opts: { cid?: string; status?: number; body?: string;
cause?: unknown } = {}) {
24             super(message);
25             this.name = "ApiError";
26             this.cid = opts.cid;
27             this.status = opts.status;
28             this.body = opts.body;
29             if (opts.cause !== undefined) (this as { cause?: unknown }).cause = opts.cause;
30         }
31     }
32
33     interface RequestOptions {
34         method?: string;
35         body?: string;
36         headers?: Record<string, string>;
37         /** Correlation id to thread through this call; auto-generated if absent. */
38         cid?: string;
39     }
40
41     async function request<T>(path: string, opts: RequestOptions = {}): Promise<T> {
42         const cid = opts.cid ?? newCorrelationId();
43         const url = `${BASE}${path}`;
44         const method = opts.method ?? "GET";
45         log("info", "api.request", { cid, method, url });
46
47         let res: Response;
48         try {
49             res = await fetch(url, {
50                 method,
51                 body: opts.body,
52                 headers: { "content-type": "application/json", "x-correlation-id": cid,
...(opts.headers ?? {}) },
53             });
54         } catch (err) {
55             // Loud failure, never silent (LOCAL_APPLIANCE_ARCHITECTURE.md §5).
56             log("error", "api.network_error", { cid, method, url, error: String(err) });
57             throw new ApiError(`Network error calling ${method} ${url}`, { cid, cause: err });
58         }
59
60         if (!res.ok) {
61             const body = await res.text().catch(() => "");
62             log("error", "api.http_error", { cid, method, url, status: res.status, body:
body.slice(0, 500) });
63             throw new ApiError(`${method} ${url} failed: ${res.status}`, { cid, status:
res.status, body });
64         }
65
66         const data = (await res.json()) as T;
67         log("info", "api.response", { cid, method, url, status: res.status });
68         return data;
69     }
70
71     /** Submit a video (a `gs://`/`gcs://`/`s3://` bucket URI, or a path on the engine host)
for
72     * processing. POST /api/process {video_path} ? 202 {job_id, ...} [main.py:260]. Fast-
fails 400
73     * (unsupported scheme / out-of-allowed-root) or 404 (missing LOCAL file) ? surfaced as
ApiError. */
74     export async function processVideo(videoPath: string, cid?: string):

```

```

Promise<ProcessResponse> {
75     return request<ProcessResponse>("/process", {
76         method: "POST",
77         body: JSON.stringify({ video_path: videoPath }),
78         cid,
79     });
80 }
81
82 /** Poll a submitted job. GET /api/jobs/{job_id} [main.py:303]. 404 ? unknown job_id. */
83 export async function getJob(jobId: string, cid?: string): Promise<JobStatus> {
84     return request<JobStatus>(`/jobs/${encodeURIComponent(jobId)}`, { method: "GET", cid
});
85 }
86
87 /** List a video's detected rallies. POST /api/query {video_id} ? {play_segments}
[main.py:331]. */
88 export async function listRallies(videoId: string, cid?: string):
Promise<RallySegment[]> {
89     const data = await request<QueryResponse>("/query", {
90         method: "POST",
91         body: JSON.stringify({ video_id: videoId }),
92         cid,
93     });
94     return data.play_segments ?? [];
95 }
96
97 /** Stitch selected rallies into one reel. POST /api/compile {video_id, segment_ids,
output_filename} [main.py:342]. */
98 export async function compileReel(
99     videoId: string,
100     segmentIds: number[],
101     outputFilename: string,
102     cid?: string,
103 ): Promise<CompileResponse> {
104     return request<CompileResponse>("/compile", {
105         method: "POST",
106         body: JSON.stringify({ video_id: videoId, segment_ids: segmentIds, output_filename:
outputFilename }),
107         cid,
108     });
109 }
110
111 /** The stream/download URL for a compiled reel. GET
/api/highlights/{video_id}/{filename} [main.py:307]. */
112 export function highlightUrl(videoId: string, filename: string): string {
113     return
`${BASE}/highlights/${encodeURIComponent(videoId)}/${encodeURIComponent(filename)}`;
114 }
115
116 /** Read engine config (for the stitching floor). GET /api/config [main.py:121]. */
117 export async function getConfig(cid?: string): Promise<EngineConfig> {
118     return request<EngineConfig>("/config", { method: "GET", cid });
119 }
120
121 /** The shot-count floor /api/compile silently enforces ? the UI must warn, not hide
[main.py:362]. */
122 export function minShotCount(cfg: EngineConfig): number | undefined {
123     return cfg.stitching?.min_shot_count;
124 }
125
126 // ?? chunked upload (backend/api/uploads.py) ?????????????????????????????????
127 // A browser uploads a LOCAL file in sequential parts ? max_part_mb (free-tier edge
proxies cap
128 // bodies ~100 MB), then `complete` returns a server-host `path` to feed straight into
/api/process.

```

```

129
130     /** Start a chunked upload. 400 = bad extension; 413 = over the size limit
[uploads.py:64]. */
131     export async function uploadInit(filename: string, totalSizeBytes: number, cid?:
string): Promise<UploadInitResponse> {
132         return request<UploadInitResponse>("/upload/init", {
133             method: "POST",
134             body: JSON.stringify({ filename, total_size_bytes: totalSizeBytes }),
135             cid,
136         });
137     }
138
139     /** Append ONE part (raw bytes ? not JSON). Parts are strictly sequential 0,1,2,?
[uploads.py:89]. */
140     export async function uploadPart(
141         uploadId: string,
142         index: number,
143         chunk: Blob,
144         cid?: string,
145         signal?: AbortSignal,
146     ): Promise<UploadPartResponse> {
147         const c = cid ?? newCorrelationId();
148         const url = `${BASE}/upload/${encodeURIComponent(uploadId)}/part/${index}`;
149         log("info", "api.request", { cid: c, method: "PUT", url });
150         let res: Response;
151         try {
152             res = await fetch(url, {
153                 method: "PUT",
154                 body: chunk,
155                 headers: { "content-type": "application/octet-stream", "x-correlation-id": c },
156                 signal,
157             });
158         } catch (err) {
159             log("error", "api.network_error", { cid: c, method: "PUT", url, error: String(err)
});
160             throw new ApiError(`Network error calling PUT ${url}`, { cid: c, cause: err });
161         }
162         if (!res.ok) {
163             const body = await res.text().catch(() => "");
164             log("error", "api.http_error", { cid: c, method: "PUT", url, status: res.status,
body: body.slice(0, 500) });
165             throw new ApiError(`PUT ${url} failed: ${res.status}`, { cid: c, status: res.status,
body });
166         }
167         log("info", "api.response", { cid: c, method: "PUT", url, status: res.status });
168         return (await res.json()) as UploadPartResponse;
169     }
170
171     /** Finalize: returns {path, video_id} ? feed `path` to processVideo() [uploads.py:123].
*/
172     export async function uploadComplete(uploadId: string, totalParts: number, cid?:
string): Promise<UploadCompleteResponse> {
173         return
request<UploadCompleteResponse>(`/upload/${encodeURIComponent(uploadId)}/complete`, {
174             method: "POST",
175             body: JSON.stringify({ total_parts: totalParts }),
176             cid,
177         });
178     }
179
180     /** Abort an in-flight upload (best-effort cleanup of the server-side partial)
[uploads.py:147]. */
181     export async function uploadAbort(uploadId: string, cid?: string): Promise<void> {
182         await request<unknown>(`/upload/${encodeURIComponent(uploadId)}`, { method: "DELETE",
cid });

```

```

183     }
184
185     export interface UploadProgress {
186         sentBytes: number;
187         totalBytes: number;
188     }
189
190     /** Upload a local File end to end: init ? sequential parts (byte progress, abortable) ?
complete.
191     * On ANY failure or abort it best-effort DELETES the server-side partial, then
rethrows so the
192     * caller can classify it (the engine's 400/413 detail survives). Returns {path,
video_id}. */
193     export async function uploadFile(
194         file: File,
195         opts: { cid?: string; signal?: AbortSignal; onProgress?: (p: UploadProgress) => void }
= {},
196     ): Promise<UploadCompleteResponse> {
197         const cid = opts.cid ?? newCorrelationId();
198         const init = await uploadInit(file.name, file.size, cid);
199         const partBytes = Math.max(1, init.max_part_mb) * 1024 * 1024;
200         const totalParts = Math.max(1, Math.ceil(file.size / partBytes));
201         log("info", "upload.start", { cid, filename: file.name, size: file.size, parts:
totalParts, part_mb: init.max_part_mb });
202         let sent = 0;
203         try {
204             for (let i = 0; i < totalParts; i++) {
205                 if (opts.signal?.aborted) throw new DOMException("Upload aborted", "AbortError");
206                 const chunk = file.slice(i * partBytes, Math.min((i + 1) * partBytes, file.size));
207                 await uploadPart(init.upload_id, i, chunk, cid, opts.signal);
208                 sent += chunk.size;
209                 opts.onProgress?.({ sentBytes: sent, totalBytes: file.size });
210             }
211             const done = await uploadComplete(init.upload_id, totalParts, cid);
212             log("info", "upload.done", { cid, video_id: done.video_id, path: done.path, size:
done.size_bytes });
213             return done;
214         } catch (err) {
215             log("warn", "upload.aborting_partial", { cid, upload_id: init.upload_id, error:
String(err) });
216             await uploadAbort(init.upload_id, cid).catch(() => {});
217             throw err;
218         }
219     }
220

```

6. assistant (2026-06-21T16:31:49.182Z)

Both files are read. The git pull failure is unrelated (a stale branch-tracking config on the engine repo); the working tree is intact. Let me verify a few load-bearing details: the part-size boundary comparison in the engine, the `/api/process` host-path acceptance in DEFAULT mode, and the SPA's error-classification/0-byte handling.

7. user (2026-06-21T16:31:50.162Z)

```

1      // Drives the in-UI file UPLOAD half of ingest (tracker T4): chunk a local file up to
the engine's
2      // /api/upload/* endpoints, report byte progress, then hand the returned server PATH to
the existing
3      // processing back-half (useIngestJob.submit). Alpha users have a FILE, not a gs:// URI
? this is the
4      // keystone that lets them point the tool at it from the browser.
5      //
6      // Kept i18n-FREE (returns stable IngestErrorCodes via classifyError, not prose) so it
unit-tests

```



```

7      // without a translation context ? IngestPanel maps codes to localized strings via
errorText. Abort
8      // is an AbortController: cancel/unmount/a superseding pick stops the in-flight upload
and the
9      // orchestrator best-effort DELETES the server-side partial.
10     import { useCallback, useEffect, useRef, useState } from "react";
11     import { uploadFile } from "../api/client";
12     import { classifyError, type IngestError } from "../lib/errors";
13     import { log, newCorrelationId } from "../lib/log";
14
15     export type UploadPhase =
16       | { status: "idle" }
17       | { status: "uploading"; pct: number; filename: string }
18       | { status: "error"; error: IngestError };
19
20     export function useUpload(onComplete: (path: string) => void) {
21       const [phase, setPhase] = useState<UploadPhase>({ status: "idle" });
22       const ctrlRef = useRef<AbortController | null>(null);
23       // Keep the latest onComplete without making start depend on it (a parent re-render
won't restart).
24       const onCompleteRef = useRef(onComplete);
25       onCompleteRef.current = onComplete;
26
27       const stop = useCallback(() => {
28         ctrlRef.current?.abort();
29         ctrlRef.current = null;
30       }, []);
31
32       // Abort any in-flight upload on unmount.
33       useEffect(() => stop, [stop]);
34
35       const start = useCallback(
36         (file: File) => {
37           // A 0-byte "video" is never valid ? reject before touching the network (reuse
invalidUri copy).
38           if (!file || file.size === 0) {
39             setPhase({ status: "error", error: { code: "invalidUri" } });
40             return;
41           }
42
43           stop(); // supersede any prior upload
44           const ctrl = new AbortController();
45           ctrlRef.current = ctrl;
46           const cid = newCorrelationId();
47           setPhase({ status: "uploading", pct: 0, filename: file.name });
48           log("info", "upload.submit_start", { cid, filename: file.name, size: file.size });
49
50           void (async () => {
51             try {
52               const res = await uploadFile(file, {
53                 cid,
54                 signal: ctrl.signal,
55                 onProgress: ({ sentBytes, totalBytes }) => {
56                   if (ctrl.signal.aborted) return;
57                   const pct = totalBytes > 0 ? Math.min(100, Math.round((sentBytes /
totalBytes) * 100)) : 0;
58                   setPhase({ status: "uploading", pct, filename: file.name });
59                 },
60               });
61               if (ctrl.signal.aborted) return;
62               setPhase({ status: "idle" }); // hand off ? the engine processing phase takes
over
63               onCompleteRef.current(res.path);
64             } catch (err) {
65               // User-cancelled (abort) is silent, not an error.

```

```

66         if (ctrl.signal.aborted || (err instanceof DOMException && err.name ===
"AbortError")) return;
67         setPhase({ status: "error", error: classifyError(err) });
68         log("error", "upload.failed", { cid, filename: file.name, error: String(err)
});
69     }
70     })();
71     },
72     [stop],
73 );
74
75     const cancel = useCallback(() => {
76         stop();
77         setPhase({ status: "idle" });
78         log("info", "upload.cancel", {});
79     }, [stop]);
80
81     const reset = useCallback(() => {
82         stop();
83         setPhase({ status: "idle" });
84     }, [stop]);
85
86     const busy = phase.status === "uploading";
87     return { phase, start, cancel, reset, busy };
88 }
89

```

8. user (2026-06-21T16:31:50.591Z)

```

1      // Types mirror the engine's verified contract (badminton-highlight-indexer).
2      // Anchors: backend/infrastructure/database.py:124-135 (play_segments),
3      // backend/main.py:331-396 (query/compile), :307-328 (highlights), :121 (config).
4
5      export interface RallyMetadata {
6          /** Gemini's shuttle_exchange_count (fallback max(3, dur/0.8)); absent on some
detectors. */
7          shot_count?: number;
8          /** Gemini confidence STRING (often "Unknown") ? not a numeric meter. */
9          shot_count_confidence?: string;
10         detector?: string;
11         [key: string]: unknown;
12     }
13
14     export interface RallySegment {
15         /** The integer the UI selects and passes to /api/compile as segment_ids. */
16         id: number;
17         start_time: number;
18         end_time: number;
19         duration: number;
20         /** motion-segmenter FABRICATION (random) ? DO NOT surface (see
PER_RALLY_SELECTION.md). */
21         server: string | null;
22         /** motion-segmenter FABRICATION (random) ? DO NOT surface. */
23         receiver: string | null;
24         metadata: RallyMetadata;
25         /** motion-only fabricated sub-events ? DO NOT surface. */
26         events?: unknown[];
27     }
28
29     export interface QueryResponse {
30         play_segments: RallySegment[];
31     }
32
33     export interface CompileResponse {
34         status: string;

```

```

35     /** server-local path (NOT a URL) ? build the download URL with highlightUrl(). */
36     filepath: string;
37 }
38
39 export interface EngineConfig {
40     stitching?: {
41         /** The shot-count floor /api/compile silently enforces ? surface it, don't hide it.
42         */
43         min_shot_count?: number;
44         [key: string]: unknown;
45     };
46     [key: string]: unknown;
47 }
48
49 /** POST /api/process ? 202 [backend/main.py:260]. The job is queued; poll GET
50 /api/jobs/{job_id}. */
51 export interface ProcessResponse {
52     job_id: string;
53     /** Always "queued" at submit. */
54     status: string;
55     /** Relative poll path, e.g. "/api/jobs/<id>". */
56     poll: string;
57 }
58
59 /** Chunked upload (backend/api/uploads.py). The browser uploads a local file in
60 sequential parts
61 * ? max_part_mb, then `complete` returns a server-host `path` to feed straight into
62 POST /api/process. */
63 export interface UploadInitResponse {
64     upload_id: string;
65     /** Per-part ceiling in MB ? slice the file into parts this big. */
66     max_part_mb: number;
67     next_part: string;
68 }
69
70 export interface UploadPartResponse {
71     upload_id: string;
72     received_parts: number;
73     received_bytes: number;
74 }
75
76 export interface UploadCompleteResponse {
77     /** Absolute path on the engine host ? pass as video_path to POST /api/process. */
78     path: string;
79     video_id: string;
80     size_bytes: number;
81 }
82
83 /** GET /api/jobs/{job_id} [backend/main.py:303]. Two status layers:
84 * `status` = the EXECUTION state; `result.status` = the pipeline VERDICT
85 (success|failed).
86 * A `done` job whose `result.status === "failed"` (or with no video_id) is a success-
87 shaped
88 * FAILURE ? the UI must not treat it as a load. */
89 export interface JobStatus {
90     job_id: string;
91     status: "queued" | "running" | "done" | "failed";
92     /** Free-text coarse phase ("hashing", "segmenting (gemini)", "finished") ? NOT a
93 percentage. */
94     progress?: string | null;
95     /** The canonical video_id, surfaced top-level on success (also in result.video_id).
96     */
97     video_id?: string | null;
98     /** Execution error (worker crash / unexpected) when status === "failed". */
99     error?: string | null;
100     result?: { status?: string; video_id?: string; message?: string } | null;
101 }

```

9. assistant (2026-06-21T16:31:54.111Z)

Now let me verify the engine's `/api/process` host-path acceptance in DEFAULT mode, the allowed_video_root / HOSTED-mode caveat, and the SPA's error classification (413 surfacing).

10. user (2026-06-21T16:31:55.066Z)

```

1      // Error helpers shared by the ingest flow and the build flow. Kept i18n-FREE: callers
map an
2      // IngestErrorCode to a localized string (so hooks stay testable without a translation
context),
3      // while the engine's own `detail` text passes through verbatim (it's the most specific
message).
4      import { ApiError } from "../api/client";
5
6      /** Pull FastAPI's {"detail": "..."} message out of an error body, else return the raw
body. */
7      export function parseDetail(body: string): string {
8          try {
9              const j = JSON.parse(body) as { detail?: unknown };
10             if (j && typeof j.detail === "string") return j.detail;
11         } catch {
12             /* not JSON ? fall through to the raw body */
13         }
14         return body;
15     }
16
17     /** Stable, localizable failure categories surfaced by the ingest flow.
18     * `engine` carries the engine's own message in `detail`; the rest are localized by
code. */
19     export type IngestErrorCode = "invalidUri" | "networkError" | "emptyResult" | "timeout"
| "engine";
20
21     export interface IngestError {
22         code: IngestErrorCode;
23         /** Present for `engine` ? the verbatim FastAPI detail / status; rendered as-is, not
translated. */
24         detail?: string;
25     }
26
27     /** Classify a thrown client error into a stable code. A network throw (ApiError with no
`status`)
28     * is the common "engine unreachable" case ? its own friendly code; any HTTP error
surfaces the
29     * engine's `detail` (e.g. an unsupported-scheme 400, a not-found 404). */
30     export function classifyError(err: unknown): IngestError {
31         if (err instanceof ApiError) {
32             if (err.status === undefined) return { code: "networkError" }; // fetch threw ?
engine truly unreachable
33             if (err.body) {
34                 const detail = parseDetail(err.body);
35                 if (detail) return { code: "engine", detail }; // a real engine message (e.g. a
400 unsupported-scheme)
36             }
37             // A bare 5xx with no body is almost always the engine being down / a gateway that
can't reach it
38             // (e.g. the dev proxy when :8000 is off) ? for a non-technical user a friendly "is
it running?"
39             // beats a cryptic "HTTP 500". A 4xx with no body keeps the status (it's a client-
side mistake).
40             if (err.status >= 500) return { code: "networkError" };
41             return { code: "engine", detail: `HTTP ${err.status}` };
42         }

```

```

43     return { code: "engine", detail: String(err) };
44 }
45

```

11. user (2026-06-21T16:31:55.412Z)

```

1     // The front-half ingest screen (appliance S1/S2; tracker B-P2 + T4). Two ways to point
the tool at a
2     // video: UPLOAD a local file (primary ? alpha users have a file, not a URI; chunked to
the engine's
3     // /api/upload/* endpoints) or paste a `gs://` bucket URI / host path (secondary). Both
converge on
4     // the SAME back-half: POST /api/process ? poll GET /api/jobs/{id} ? hand the video_id
to App. Honest
5     // by construction: the engine reports coarse phase WORDS (not a %), and a success-
shaped failure
6     // (done but no rallies) is shown, never silently treated as a load.
7     import { useState } from "react";
8     import type { ChangeEvent, FormEvent } from "react";
9     import { useTranslation } from "react-i18next";
10    import { useIngestJob } from "../hooks/useIngestJob";
11    import { useUpload } from "../hooks/useUpload";
12    import { errorText } from "../lib/errorText";
13    import { StatusChip, Spinner } from "../StatusChip";
14
15    interface Props {
16        /** Called with the engine's video_id once a submitted job finishes successfully. */
17        onLoaded: (videoId: string) => void;
18    }
19
20    export function IngestPanel({ onLoaded }: Props) {
21        const { t } = useTranslation();
22        const [uri, setUri] = useState("");
23        const { phase, submit, cancel: cancelIngest, reset: resetIngest, busy: ingestBusy } =
useIngestJob(onLoaded);
24        // On upload completion, feed the returned server PATH straight into the engine
processing back-half.
25        const upload = useUpload((path) => submit(path));
26        const busy = ingestBusy || upload.busy;
27
28        const onSubmitUri = (e: FormEvent) => {
29            e.preventDefault();
30            submit(uri);
31        };
32        const onFile = (e: ChangeEvent<HTMLInputElement>) => {
33            const file = e.target.files?.[0];
34            if (file) upload.start(file);
35            e.target.value = ""; // let re-picking the same file fire change again
36        };
37        const cancelAll = () => {
38            upload.cancel();
39            cancelIngest();
40        };
41        const resetAll = () => {
42            upload.reset();
43            resetIngest();
44        };
45
46        // Shared mapping with the build flow (lib/errorText): an `engine` failure carries the
engine's own
47        // message (a 400 scheme/path/extension detail, a worker error); every other code maps
to a friendly
48        // localized line. Surface whichever half failed (upload or engine) ? they're mutually
exclusive.
49        const errorState = upload.phase.status === "error" ? upload.phase.error : phase.status

```

```

=== "error" ? phase.error : null;
50     const errorMsg = errorState ? errorText(t, errorState) : null;
51
52     return (
53         <section className="mt-6 rounded-2xl border border-black/10 bg-white p-5" data-
testid="ingest-panel">
54             <h2 className="text-lg font-bold">{t("ingest.title")}</h2>
55             <p className="mt-1 max-w-2xl text-sm text-ink/60">{t("ingest.intro")}</p>
56
57             {/* PRIMARY: upload a local file (T4). A native file input, visually a button
(?44px, A4). */}
58             <div className="mt-4">
59                 <label
60                     className={
61                         "inline-flex min-h-11 items-center rounded-xl bg-ink px-4 py-2 text-sm font-
semibold text-white hover:bg-ink2 " +
62                         (busy ? "pointer-events-none opacity-40" : "cursor-pointer")
63                     }
64                 >
65                     {t("ingest.chooseFile")}
66                     <input
67                         type="file"
68                         accept=".mp4,.mov,video/mp4,video/quicktime"
69                         aria-label={t("ingest.uploadAria")}
70                         disabled={busy}
71                         onChange={onFile}
72                         className="sr-only"
73                     />
74                 </label>
75                 <p className="mt-2 text-xs text-ink/40">{t("ingest.fileHint")}</p>
76             </div>
77
78             {/* SECONDARY: or point at a cloud bucket / host path. */}
79             <div className="mt-4 border-t border-black/5 pt-4">
80                 <p className="text-xs font-semibold uppercase tracking-wide text-
ink/40">{t("ingest.orLink")}</p>
81                 <form onSubmit={onSubmitUri} className="mt-2 flex flex-wrap items-center gap-2">
82                     <input
83                         type="text"
84                         value={uri}
85                         onChange={(e) => setUri(e.target.value)}
86                         placeholder={t("ingest.uriPlaceholder")}
87                         aria-label={t("ingest.uriAriaLabel")}
88                         disabled={busy}
89                         className="min-h-11 min-w-[18rem] flex-1 rounded-xl border border-black/15
bg-white px-4 py-2 text-sm text-ink outline-none focus:border-green disabled:opacity-50"
90                     />
91                     <button
92                         type="submit"
93                         disabled={busy}
94                         className="min-h-11 rounded-xl bg-ink px-4 py-2 text-sm font-semibold text-
white disabled:opacity-40"
95                     >
96                         {phase.status === "submitting" ? t("ingest.submitting") :
t("ingest.submit")}
97                     </button>
98                 </form>
99                 <p className="mt-2 text-xs text-ink/40">{t("ingest.hint")}</p>
100             </div>
101
102             {/* Live status (aria-live so screen readers hear progress) as icon+text chips ?
never icon-only
103             (A8). Upload shows a determinate % bar; the engine phase is indeterminate
(phase words). */}
104             <div aria-live="polite" className="mt-3 text-sm" data-testid="ingest-status">

```

```

105         {upload.phase.status === "uploading" && (
106             <div data-testid="upload-progress">
107                 <StatusChip tone="info" icon={<Spinner />} label={t("ingest.uploading", {
108                     pct: upload.phase.pct })} />
109                 <div
110                     className="mt-2 h-1.5 w-full max-w-sm overflow-hidden rounded-full bg-
black/10"
111                     role="progressbar"
112                     aria-valuenow={upload.phase.pct}
113                     aria-valuemin={0}
114                     aria-valuemax={100}
115                 >
116                     <div className="h-full bg-green transition-all" style={{ width:
` ${upload.phase.pct}% ` }} />
117                 </div>
118                 <p className="mt-2 text-xs text-ink/50">{t("ingest.reassure")}</p>
119                 <button
120                     type="button"
121                     onClick={cancelAll}
122                     className="mt-2 inline-flex min-h-11 items-center rounded-xl border
border-black/15 px-4 text-sm hover:border-green"
123                 >
124                     {t("ingest.cancel")}
125                 </button>
126             </div>
127         )}
128         {ingestBusy && (
129             <div>
130                 <StatusChip
131                     tone="info"
132                     icon={<Spinner />}
133                     label={
134                         t("ingest.working") +
135                         (phase.status === "polling" && phase.progress
136                             ? ` · ${t("ingest.progress", { progress: phase.progress })}`
137                             : "")
138                     }
139                 />
140                 {/* Reassurance for slow uplinks: the job runs on the server, so it's safe
to wait. */}
141                 <p className="mt-2 text-xs text-ink/50">{t("ingest.reassure")}</p>
142                 <button
143                     type="button"
144                     onClick={cancelAll}
145                     className="mt-2 inline-flex min-h-11 items-center rounded-xl border
border-black/15 px-4 text-sm hover:border-green"
146                 >
147                     {t("ingest.cancel")}
148                 </button>
149             </div>
150         )}
151         {errorMsg && (
152             <div className="rounded-lg border border-red-200 bg-red-50 px-3 py-2 text-
red-800" data-testid="ingest-error">
153                 <StatusChip tone="error" icon="?" label={t("ingest.statusFailed")} />
154                 <p className="mt-2">{errorMsg}</p>
155                 <button
156                     type="button"
157                     onClick={resetAll}
158                     className="mt-2 inline-flex min-h-11 items-center rounded-lg border
border-red-300 px-3 text-sm hover:bg-red-100"
159                 >
160                     {t("ingest.retry")}
161                 </button>
162             </div>

```

```

162         })
163     </div>
164 </section>
165 );
166 }
167

```

12. assistant (2026-06-21T16:31:59.871Z)

Now let me verify the engine's `/api/process` host-path acceptance in DEFAULT mode and the allowed_video_root / HOSTED-mode caveat. Let me look at the engine's process endpoint and path-resolution.

13. user (2026-06-21T16:32:01.374Z)

```

240         return response
241
242         logger.info(f"[API] Using segmenter: {seg_name}")
243         notify(f"segmenting ({seg_name})")
244         segmenter_actual = seg_name
245         # Destroy-AFTER-confirm: remember the pre-run rows, then keep new + drop old
246         only
247         # if the run yields segments; otherwise drop the new partial rows and keep the
248         old.
249         play_wm, cand_wm = db_instance.segment_watermarks(video_id)
250         segmenter = segmenter_cls(db_instance, global_config)
251         try:
252             result = segmenter.process_video(video_id, local_video, req.player_pool,
253             req.max_frames)
254         except Exception:
255             # A raising segmenter must not leave half-written rows: restore the pre-run
256             # state (same destroy-after-confirm contract as the Failure branch), then
257             let
258             # the job worker record the failure.
259             db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
260             raise
261         segmentation_ran = True
262
263         if isinstance(result, Failure):
264             logger.error(f"[API] Segmenter failed: {result.message}")
265             db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
266             response = {
267                 "video_id": video_id,
268                 "status": "failed",
269                 "message": f"Segmenter '{seg_name}' failed: {result.message}",
270                 "results": [r.model_dump() for r in val_results],
271                 "play_segments": [],
272             }
273             return response
274
275         segments = result.value if hasattr(result, "value") else result
276         notify("finalizing")
277         _finalize_reingest(video_id, segments, play_wm, cand_wm)
278
279         response = {
280             "video_id": video_id,
281             "status": "success",
282             "message": f"Successfully indexed {len(segments)} play segments using
283             '{seg_name}' segmenter.",
284             "results": [r.model_dump() for r in val_results],
285             "play_segments": segments,
286         }
287         return response
288     finally:
289         # Always emit the per-video observability report (next to the video, or to

```



```

285         # report.output_dir). Never raises. Surface the paths in the response.
286         paths = emit_indexer_report(
287             db=db_instance, video_id=video_id, video_path=req.video_path,
config=global_config,
288             val_results=val_results, val_context=val_context,
289             segmenter_requested=req.segmenter, segmenter_actual=segmenter_actual,
290             was_reingest=was_reingest, segmentation_ran=segmentation_ran,
291         )
292         if paths and isinstance(response, dict):
293             response["reports"] = paths
294
295
296     @app.post("/api/process", status_code=202)
297     def process_video(req: ProcessRequest, request: Request):
298         """Submit a processing job (202 + job_id). Poll GET /api/jobs/{job_id} for state.
299
300         Fast-fail checks (bad path, missing file, unknown segmenter) happen here so the
301         client gets an immediate 4xx; everything slow ? including MD5 hashing of the file ?
302         runs on the worker (DEFERRED_HARDENING_PLAN #16). The row is created 'queued'; the
303         worker drains it via `claim_due_job` (EXECUTION_POLICY.md P3 self-scheduling loop).
304         """
305         # M9 (D9): edge-auth tenant resolution. DEFAULT-OFF ? owner_id None (the local
single-user,
306         # unchanged). When edge_auth_enabled, the Cf-Access JWT is required + verified (a
missing or
307         # spoofed identity header fails here with 401, before any work) and tags the job's
owner_id.
308         try:
309             owner_id = edge_auth.resolve_tenant(request.headers, global_config)
310         except edge_auth.EdgeAuthError as e:
311             raise HTTPException(status_code=401, detail=f"edge auth: {e}") from e
312
313         allowed_root = getattr(getattr(global_config, "security", None),
"allowed_video_root", None)
314         try:
315             validate_input_path(req.video_path, allowed_root=allowed_root)
316             # M2: resolve the input once (path or storage URI) ? raises ValueError for an
unsupported
317             # scheme, which we map to a clean 400 (an unknown scheme must never 500).
318             input_ref = storage.resolve_input_ref(req.video_path, getattr(global_config,
"storage", None))
319         except ValueError as e:
320             raise HTTPException(status_code=400, detail=str(e)) from e
321         # The local-existence fast-fail applies only to a LOCAL input, and stats the
RESOLVED local path
322         # (input_ref.key ? e.g. local://? stripped to the bare path), not the raw URI
string. A
323         # bucket/Drive URI can't be cheaply os.path.exists()'d ? its existence is verified
when the worker
324         # fetches it (a fetch miss fails the job loudly). validate_input_path above still
runs for every
325         # input: its null-byte guard always applies, and a configured allowed_video_root
(hosted mode)
326         # rejects a non-local URI as out-of-root.
327         if input_ref.backend == "local" and not os.path.exists(input_ref.key):
328             raise HTTPException(status_code=404, detail=f"Video file not found at
'{req.video_path}'")
329         if req.segmenter and not segmenter_registry.get(req.segmenter):

```

14. user (2026-06-21T16:32:02.434Z)

C:\Users\avidu\Projects\badminton-highlight-indexer\backend\api\uploads.py:3:Browser uploads arrive in sequential parts ? security.max_part_mb (free-tier edge
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\api\uploads.py:5:under security.upload_dir, then submitted to /api/process by the returned path.

```

C:\Users\avidu\Projects\badminton-highlight-indexer\backend\api\uploads.py:7: aborts partial
uploads and the client re-inits. Per-user partitioning of upload_dir
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\api\uploads.py:47:     upload_dir =
getattr(sec, "upload_dir", None) or "output/uploads"
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\api\uploads.py:48:     max_bytes =
int(getattr(sec, "max_upload_mb", 4096) or 4096) * 1024 * 1024
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\api\uploads.py:49:     part_bytes =
int(getattr(sec, "max_part_mb", 95) or 95) * 1024 * 1024
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\api\uploads.py:51:     return
upload_dir, max_bytes, part_bytes, exts
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\api\uploads.py:67:     upload_dir,
max_bytes, part_bytes, exts = _upload_cfg()
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\api\uploads.py:79:
os.makedirs(upload_dir, exist_ok=True)
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\api\uploads.py:81:     tmp_path =
os.path.join(upload_dir, f".partial-{upload_id}")
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\api\uploads.py:85:     return
{"upload_id": upload_id, "max_part_mb": part_bytes // (1024 * 1024),
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\api\uploads.py:126:     upload_dir,
_, _, _ = _upload_cfg()
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\api\uploads.py:134:     final_path =
os.path.join(upload_dir, st["filename"])
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\api\uploads.py:137:
final_path = os.path.join(upload_dir, f"{stem}-{upload_id[:8]}{ext}")
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\main.py:313:     allowed_root =
getattr(getattr(global_config, "security", None), "allowed_video_root", None)
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\main.py:325:     # input: its null-
byte guard always applies, and a configured allowed_video_root (hosted mode)
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\main.py:583:     auth on without the
`allowed_video_root` jail / CF team+AUD). A strict no-op when no
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\startup.py:22: rest of the
posture at **boot**: the `allowed_video_root` path-jail is set (else an authenticated
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\startup.py:103:     # a tenant
can point the engine at an ARBITRARY local file (allowed_video_root unset = "any local
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\startup.py:105:     root =
getattr(sec, "allowed_video_root", None)
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\startup.py:109:
"security.edge_auth_enabled is ON (engine exposed) but security.allowed_video_root is "
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\startup.py:111:
"Set allowed_video_root to a jail directory before exposing the engine.",
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\models.py:283:
allowed_video_root: Optional[str] = None
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\models.py:287:     # arrive
with PR-3 identity scoping. ? In hosted mode, set allowed_video_root to
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\models.py:288:     # contain
upload_dir or /api/process will reject the uploaded paths.
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\models.py:289:     upload_dir:
str = "output/uploads"
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\models.py:292:
max_upload_mb: int = 4096
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\models.py:293:
max_part_mb: int = 95

```

15. assistant (2026-06-21T16:32:06.098Z)

Now let me verify the critical part-size boundary: the engine uses `>` (strict) at line 106, so a part exactly equal to `part\_bytes` is accepted. Let me also confirm `validate\_input\_path` behavior when `allowed\_root` is None (DEFAULT mode). Let me check `validate\_input\_path`.

16. user (2026-06-21T16:32:07.166Z)

Found 1 file

```
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\utils\paths.py
```

17. user (2026-06-21T16:32:11.006Z)

```
1      """Path-safety helpers for the API boundary (untrusted request fields)."""
2      import os
3      from typing import Any, Optional
4
5
6      def output_base(config: Any) -> str:
7          """The configured base dir for a segmenter's LOCAL scratch/staging artifacts
8          (COMPUTE_DECOUPLED_SERVING M3). Returns ``output.output_dir`` (default ``"output"``)
9          so the
10          chunk-staging / AI-handoff / temp dirs follow ``--output-dir`` /
11          ``output.output_dir`` instead of
12          a hardcoded ``./output``. The DEFAULT is ``"output"`` ? byte-identical to the prior
13          literal.
14          This is the LOCAL output dir, deliberately NOT the storage ``workspace_root`` (which
15          may be a
16          ``gs://`` URI you can't ``mkdirs``). Accepts the typed AppConfig or the legacy
17          dict-shim
18          (``.get("output", {})).get("output_dir")``); any miss or empty value falls back to
19          ``"output"``. """
20          try:
21              out = config.get("output", {}).get("output_dir")
22          except Exception:
23              out = None
24          return out or "output"
25
26      def safe_output_filename(name: str) -> str:
27          """Validate an output filename is a BARE name (no dir part, no traversal, not
28          absolute).
29          `os.path.join(output_dir, name)` silently discards `output_dir` when `name` is
30          absolute,
31          and ``..`` segments escape it ? so an unchecked `output_filename` is an arbitrary-
32          write
33          primitive. Returns the name unchanged when safe; raises ValueError otherwise.
34          """
35          if not name or not name.strip():
36              raise ValueError("output_filename must not be empty")
37          if "\x00" in name:
38              raise ValueError("output_filename must not contain a null byte")
39          if "/" in name or "\\" in name:
40              raise ValueError(f"output_filename must not contain a path separator: {name!r}")
41          if os.path.isabs(name) or name != os.path.basename(name) or name in (".", ".."):
42              raise ValueError(f"output_filename must be a bare filename, got: {name!r}")
43          return name
44
45      def resolve_under_root(path: str, root: str) -> str:
46          """Return the real path, asserting it is contained under `root` (raises ValueError
47          if not). """
48          rp = os.path.realpath(path)
49          rr = os.path.realpath(root)
50          try:
51              contained = os.path.commonpath([rp, rr]) == rr
52          except ValueError: # different drives on Windows
53              contained = False
54          if not contained:
55              raise ValueError(f"path {path!r} escapes the allowed root {root!r}")
56          return rp
57
58      def validate_input_path(path: str, allowed_root: Optional[str] = None) -> None:
```

```

54         """Reject null bytes and (when an allowed_root is configured) out-of-root inputs.
55
56         With no allowed_root (default, single-user local mode) only the null-byte check
57         applies,
58         so the local 'point it at any of my videos' workflow is preserved. Setting an
59         allowed_root (a future hosted/tenant deployment) enforces containment.
60         """
61         if "\\x00" in path:
62             raise ValueError("video_path must not contain a null byte")
63         if allowed_root:
64             resolve_under_root(path, allowed_root)

```

18. assistant (2026-06-21T16:32:33.323Z)

I now have everything needed to verify the full contract. Let me confirm the engine's filename safety doesn't reject the SPA's filenames, and verify the part-boundary math definitively.

Key boundary check (uploads.py:106): ``if received > part_bytes:` ? strict `>``. A part exactly equal to ``part_bytes`` is accepted. The SPA's ``partBytes = max_part_mb * 1024 * 1024`` (client.ts:199) equals the engine's ``part_bytes`` exactly (uploads.py:49). Since ``file.slice`` produces parts of at most ``partBytes`` bytes, ``received`` never exceeds ``part_bytes``, so no spurious 413. Confirmed correct.

Let me check one more thing ? ``safe_output_filename`` is applied to the upload filename at init (uploads.py:69), and it rejects path separators. A browser ``file.name`` is a bare basename (browsers strip path), so that's fine. But let me note: the SPA sends ``file.name`` raw. Let me verify there's no client-side sanitization gap worth flagging, and confirm the 413/400 surface path through ``classifyError``.

The contract verification is complete. Let me compile findings.

The contract matches across all five checkpoints:

1. **init body keys + honored max_part_mb**: client.ts:131-137 sends ``{filename, total_size_bytes}`` ? exact match to ``UploadInitRequest`` (uploads.py:32-34). client.ts:199 reads ``init.max_part_mb`` from the response (not hardcoded), and the comment block at uploads.py:85 returns ``max_part_mb: part_bytes // (1024*1024)``. Honored correctly.
2. **part boundary**: client.ts:199 ``partBytes = max_part_mb*1024*1024`` equals engine ``part_bytes`` (uploads.py:49). Engine uses strict ``>`` at uploads.py:106 (``if received > part_bytes``), so a full part exactly equal to the limit passes. ``file.slice`` produces parts ? ``partBytes``. No spurious 413. PUT is raw ``octet-stream`` body (client.ts:152-156), sequential 0,1,2 (client.ts:204-210), matching ``request.stream()`` + sequential index check (uploads.py:97-99).
3. **complete total_parts**: client.ts:211 sends ``uploadComplete(init.upload_id, totalParts, ?)`` where ``totalParts`` is the loop bound (client.ts:200) = exactly the number of parts PUT. Engine 409s only on mismatch (uploads.py:131). Match.
4. **process host path**: returned ``path`` (abspath, uploads.py:142) fed via ``submit(path)`` ? ``processVideo`` ? ``{video_path}`` (client.ts:74-79). In DEFAULT mode ``allowed_root`` is None (main.py:313), ``validate_input_path`` only does the null-byte check (paths.py:62), so a host path is accepted. Confirmed.
5. **size limits / 413**: init 413s if ``total_size_bytes > max_bytes`` (uploads.py:76-78); part 413s on overflow. SPA surfaces these via ``classifyError`` ? ``code:"engine"`` carrying the verbatim FastAPI ``detail`` (errors.ts:33-35), rendered in IngestPanel (line 153). 0-byte rejected pre-network (useUpload.ts:38). Clean.

I'll record one minor observation about the HOSTED-mode caveat (a T2 deployment note, not a T4 blocker) and one nit about a friendlier client-side size guard.

19. user (2026-06-21T16:32:54.494Z)

Output does not match required schema: root: must have required property 'findings'

20. assistant (2026-06-21T16:32:58.770Z)

The schema requires `findings` and `summary` as direct properties. I passed them correctly but the tool nested them. Let me retry with the correct top-level shape.

21. user (2026-06-21T16:33:19.837Z)

Output does not match required schema: root: must have required property 'findings'

22. assistant (2026-06-21T16:33:23.344Z)

The `findings` array must be passed as a JSON array value, not a string. Let me retry with the proper structure.

23. user (2026-06-21T16:33:40.921Z)

Output does not match required schema: root: must have required property 'findings'

24. assistant (2026-06-21T16:33:45.335Z)

The validator says the root object lacks `findings`. It seems the harness may be interpreting my `findings` parameter as a string rather than parsing the JSON array. Let me pass `findings` as actual structured array values per the schema (the schema expects an array of objects).

25. user (2026-06-21T16:34:01.134Z)

Output does not match required schema: root: must have required property 'findings'

26. assistant (2026-06-21T16:34:05.616Z)

The validator keeps reporting the root lacks `findings` even though I'm providing it. The multi-line JSON in the array value may be the problem ? the harness might be failing to parse it. Let me try with `findings` as a compact single-line JSON array string with no embedded newlines.

27. user (2026-06-21T16:34:19.601Z)

Output does not match required schema: root: must have required property 'findings'

28. assistant (2026-06-21T16:34:23.746Z)

The validator persistently reports the root lacks `findings`. Since `summary` alone isn't being flagged, the harness may be having trouble parsing the `findings` argument. Let me try the simplest possible valid shape ? an empty findings array ? to confirm the mechanism works, then I'll know whether the issue is the array content.

29. user (2026-06-21T16:34:24.042Z)

Structured output provided successfully

30. assistant (2026-06-21T16:34:28.257Z)

The empty array works ? so the issue was that my `findings` array content was being rejected (likely the JSON parsing of the larger payload). The mechanism accepts a compact array. Let me now submit the real findings as a properly-formed compact array.

31. user (2026-06-21T16:34:44.247Z)

Output does not match required schema: root: must have required property 'findings'

32. assistant (2026-06-21T16:34:56.236Z)

The empty-array case passed, but the populated array fails. The difference must be a character in my content that breaks the harness's JSON parsing ? most likely the unescaped forward slashes

or the colon/special chars. Let me test with a minimal single finding to isolate.

33. user (2026-06-21T16:34:57.035Z)

Structured output provided successfully

34. assistant (2026-06-21T16:35:02.363Z)

A minimal finding works. So the parse breaks on something in my longer content. The likely culprits are the colons inside `detail`/`location` strings or the parentheses ? but JSON handles those. More likely it's the harness passing the whole tool-input as one JSON and a stray character (e.g. the `400 escapes-the-allowed-root:` colon is fine). Let me test incrementally ? add the `location` and `suggestion` fields with my real long text but no special framing.

35. user (2026-06-21T16:35:17.820Z)

Output does not match required schema: root: must have required property 'findings'

36. assistant (2026-06-21T16:35:22.175Z)

Still failing with the full content, but the minimal version passed. The breakage is content-length or a specific token. Let me try just the two findings but with much shorter `summary` to isolate whether it's total payload size.

37. user (2026-06-21T16:35:29.351Z)

Output does not match required schema: root: must have required property 'findings'

38. assistant (2026-06-21T16:35:34.865Z)

Short summary still fails with the two full findings ? so it's the findings content, not total size. The minimal finding passed. Let me submit each finding more tersely; likely a specific substring (perhaps the semicolons in `location`, or `400`, or the parenthetical chains) is tripping the harness's parser. Let me try the two findings with terse, plain detail text and no semicolons in location.

39. user (2026-06-21T16:35:39.520Z)

Structured output provided successfully

40. assistant (2026-06-21T16:35:51.870Z)

The terser version succeeded. The earlier failures were caused by some token in the longer `detail` strings that broke the harness's argument parsing. The structured output is now submitted with both findings preserved (just more tersely worded). My review is complete.

Summary of the T4 engine-contract fidelity review:

****VERIFIED CLEAN**** ? the SPA chunked-upload client matches the engine contract exactly on all five checkpoints:

1. ****init**** ? `client.ts:131-137` sends `{filename, total\_size\_bytes}` matching `UploadInitRequest` (`uploads.py:32-34`); honors the ****returned**** `max\_part\_mb` (`client.ts:199`), not a hardcoded size.
2. ****part**** ? SPA `partBytes = max\_part\_mb\*1024\*1024` (`client.ts:199`) equals engine `part\_bytes` (`uploads.py:49`); engine uses ****strict >***** at `uploads.py:106`, so a full part exactly equal to the limit is accepted (no spurious 413). Raw `octet-stream` PUT (`client.ts:152-156`), strictly sequential 0,1,2 matching the index check (`uploads.py:97-99`).
3. ****complete**** ? `total\_parts` sent (`client.ts:200,211`) equals the parts actually PUT; no 409 mismatch (`uploads.py:131`).
4. ****process**** ? returned abspath `{path}` (`uploads.py:142`) is fed to `/api/process {video\_path}` (`client.ts:74-79`); in DEFAULT mode `allowed\_root` is None (`main.py:313`) so `validate\_input\_path` only null-byte-checks (`paths.py:60-63`) and a host path is accepted.

5. **size/413** ? init 413 (`uploads.py:76-78`) and per-part 413 surface via `classifyError` as `code:"engine"` with the verbatim FastAPI detail (`errors.ts:33-35`, rendered `IngestPanel.tsx:153`); 0-byte rejected pre-network (`useUpload.ts:38`).

No blockers, no majors. Two non-blocking findings recorded: (minor) the HOSTED-mode `allowed_video_root`-must-contain-`upload_dir` caveat ? a T2 deployment note, not a T4 blocker; and (nit) the SPA has no proactive client-side max-size hint since `max_upload_mb` isn't exposed (the init 413 already prevents wasted bytes, so this is optional).